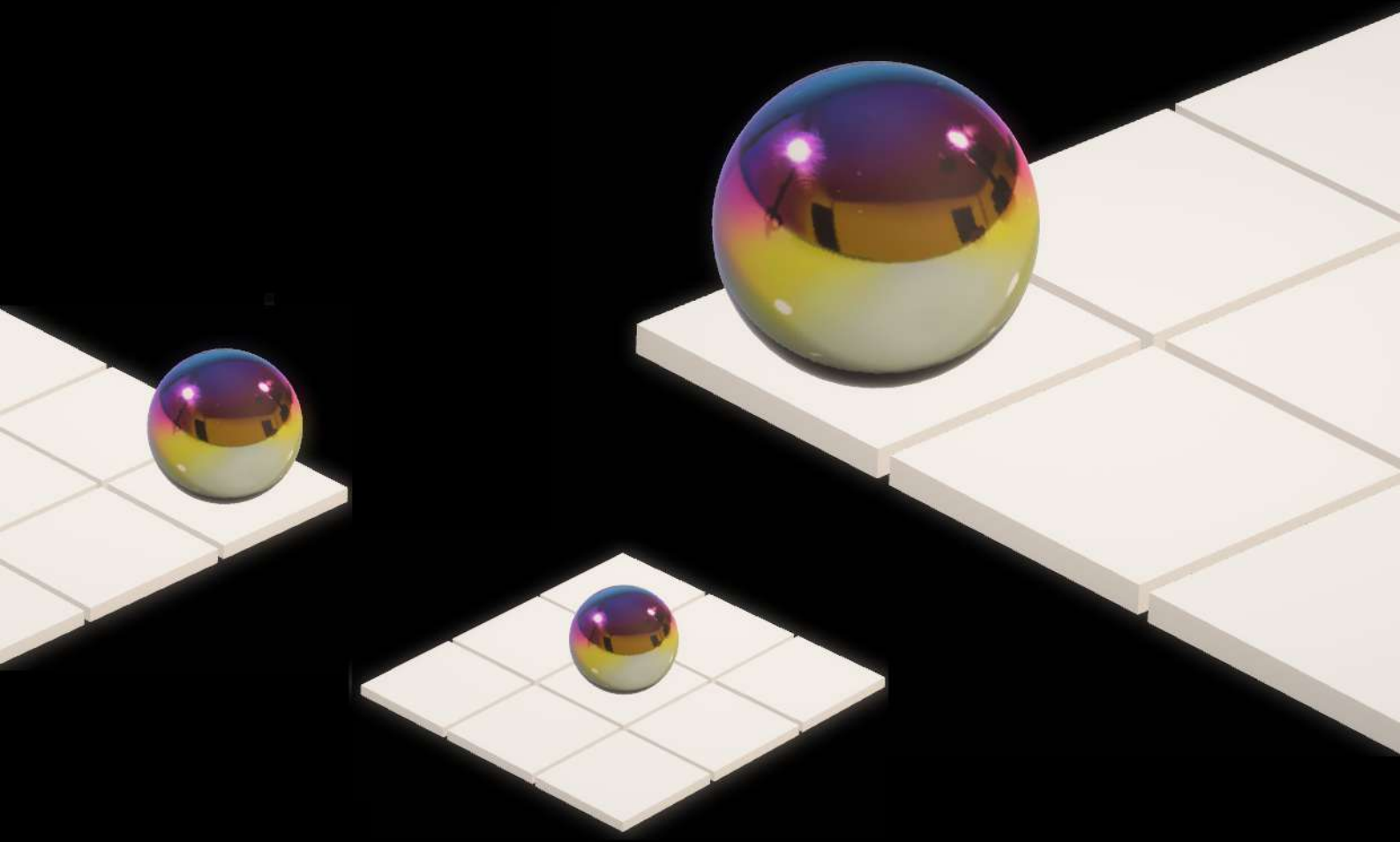
LOGIC COMPONENTS

LUKE NEO-SHELL



LUKE'S.CORP

NEO-Shell

Guida Completa ai Comandi

Logic-Software-NEO-Shell® Version I — Introduzione comandi

Logic Components Corp · By Fogli Luca · 2024 – 2025

Indice dei Capitoli

1. Introduzione e Struttura del Linguaggio
2. Variabili — var / killvar / allvar
3. Espressioni e Calcoli — $ (con tutte le funzioni matematiche)
4. Lettura Input — in D / A / K / T
5. Output Stampa — print / wait
6. Blocchi e Funzioni — blk / vob
7. Controllo del Flusso — if / elseif / else
8. Cicli — for / while / loop / switch
9. Segnali di Controllo — break / continue / return
10. Porte I/O — out d / a / t / s / p
11. EEPROM — eepset / eeplist / eeprun / eepcls / eepmem
12. Protocollo I2C — iset / ifind
13. Variabili di Sistema — NEOin / return
14. Comandi Speciali — boot / Empty
15. Riferimento Rapido Completo
A. Appendice A — Messaggi di Sistema (MSGbox)

1. Introduzione e Struttura del Linguaggio

NEO-Shell è un linguaggio di scripting interpretato progettato per il microcontrollore Atemga 1284P. Esso, permette di eseguire comandi in tempo reale dalla porta seriale a 115200 baud o tramite bus I2C, salvare programmi persistenti nella EEPROM del dispositivo, e gestire variabili, cicli, condizioni, ingressi/uscite hardware.

Sintassi di base

Il punto e virgola (;) e' un SEPARATORE tra comandi sulla stessa riga, NON un terminatore obbligatorio. Un comando da solo non ha bisogno del punto e virgola. I blocchi di codice si racchiudono tra { }. Le condizioni tra ( ). I valori di blocco tra [ ].

```
out d 13 1                          // comando singolo: nessun ; necessario
out d 13 1 ; wait 500 ; out d 13 0  // piu' comandi sulla stessa riga: separati da ;
var A = 10; var B = 20; print $A    // altro esempio di separazione
if ($A > 5) { print "ok\n" }       // blocco: anche qui il ; e' opzionale
```

Regole fondamentali

Regola	Spiegazione
`Separatore ;`	Il ; separa comandi sulla stessa riga. Non e' obbligatorio per un comando singolo.
`Variabili con $`	Per leggere il valore di una variabile usare sempre $NOME
`Blocchi { }`	Delimitano il corpo di if, for, while, loop, switch
`Condizioni ( )`	Racchiudono la condizione di if, while, for, switch
`funzione blocco [ ]`	Racchiudono il corpo di comandi in un blocco: blk name = [funzione]
`Case sensitivity`	I nomi di variabili e comandi sono case-sensitive
`Max variabili`	256 variabili simultanee in memoria
`Max stringa I2C`	32 byte per singolo messaggio I2C (limite hardware)

2. Variabili — var / killvar / allvar

var — Dichiarazione e assegnazione di variabili

Crea una nuova variabile o aggiorna il valore di una esistente. Il tipo di variabile viene rilevata automaticamente dal sistema.

Tipo rilevato	Condizione
`int`	Il valore è un numero intero (es: 10, -3, 0)
`float`	Il valore ha parte decimale (es: 3.14, -0.5)
`char`	Il valore è un singolo carattere (es: A, z, !)
`string`	Il valore è qualsiasi altra stringa (es: Hello world!, on)

Sintassi	Descrizione
`var NOME = valore`	Crea variabile con valore diretto
`var A = 10, B = 2.05, C = !`	Assegnazione multipla sulla stessa riga (separata da virgola)
`var A = $B`	Copia il valore di un'altra variabile
`var $PUNTATORE = valore`	Assegna al nome contenuto nella variabile PUNTATORE (indirezione)

Esempi

```
var Temperatura = 23;              // int
var Tensione = 3.14;               // float
var Lettera = A;                   // char
var Messaggio = CiaoMondo;         // string
var X = 10, Y = 20, Z = 0;        // multipla
var Doppio = $X;                   // copia X in Doppio
var Nome = pippo;
var $Nome = 42;                    // crea/aggiorna 'pippo' = 42  (indirezione)
```

killvar — Eliminazione di variabili dalla memoria

Elimina una o più variabili. Senza argomenti cancella tutto. Supporta lista con virgola e range con @.

Sintassi	Descrizione
`killvar`	Elimina TUTTE le variabili (reset)
`killvar NOME`	Elimina la variabile NOME
`killvar A,B,C`	Elimina A, B e C
`killvar A@Z`	Elimina tutte le var da A a Z nell'ordine di creazione

```
killvar                  // reset totale
killvar Temp             // elimina Temp
killvar A,B,C            // elimina tre variabili
```

`allvar` — Visualizzazione variabili in memoria

Stampa le variabili allocate in memoria. ATTENZIONE: il formato dell'output dipende dalla modalità di chiamata.

Sintassi	Output	Descrizione
`allvar`	NOME = valore	Mostra TUTTE le variabili nel formato 'nome = valore'
`allvar NOME`	valore	Stampa SOLO IL VALORE di NOME (senza il nome)
`allvar A,B,C`	valore\nvalore\nvalore	Stampa solo i valori di A, B, C uno per riga
`allvar A@Z`	valore\n...	Stampa solo i valori nell'intervallo A-Z (per ordine di creazione)

⚠ *Con parametro (nome, lista, range) allvar stampa SOLO il valore — senza il nome. Solo allvar senza argomenti mostra 'nome = valore'.*

```
allvar              // NOME = valore  per tutte
allvar X            // stampa: 42  (solo il valore di X)
allvar A@C          // stampa i valori di A, B, C (uno per riga)
```

3. Espressioni e Calcoli ONE FOR ALL! — “$”

Il $ è possibile svolgere un'espressione matematica/logica e assegna il risultato a una variabile. Supporta operatori, funzioni matematiche, espressioni con parentesi annidate, lettura variabili con $NOME e specifica del tipo risultato con un suffisso.

```
$ NOME = espressione;
```

Operatori aritmetici e bitwise

Operatore	Descrizione	Esempio	Risultato
+	Addizione	$A = 5 + 3	8
-	Sottrazione	$A = 10 - 4	6
*	Moltiplicazione	$A = 3 * 4	12
/	Divisione	$A = 10 / 4 f	2.5
%	Modulo (resto)	$A = 10 % 3	1
@	Potenza (right-assoc.)	$A = 2@10	1024
&	AND bit a bit	$A = 1 & 0	0
|	OR bit a bit	$A = 1 | 0	1
^	XOR bit a bit	$A = 1 ^ 1	0
~	NOT bit a bit (prefisso unario)	$A = ~0	-1
!	NOT logico (prefisso unario)	$A = !0	1
++	Incremento	$A++	A = A+1
--	Decremento	$A--	A = A-1

Operatori speciali di clamp / confronto interno

ATTENZIONE: all'interno delle espressioni $ gli operatori > < = non restituiscono 0/1 come nei linguaggi classici. Hanno comportamento di 'clamp numerico':

Operatore	Comportamento effettivo	Esempio	Risultato
A > B	Restituisce B se A > B, altrimenti A (clamp alto)	$R = 100 > 50	50
A < B	Restituisce B se A < B, altrimenti A (clamp basso)	$R = 10 < 50	50
A = B	Restituisce B se A == B, altrimenti A	$R = 5 = 5	5

⚠ *Per confronti booleani reali (true/false) usare le strutture if ($A > $B). I simboli > < = usati FUORI da if/while sono operatori di clamp, non di confronto.*

Funzioni matematiche

Funzione	Descrizione	Esempio
`$var = sin N`	Seno di N gradi	$R = sin90 → 1
`$var = cos N`	Coseno di N gradi	$R = cos0 → 1
`$var = tan N`	Tangente di N gradi	$R = tan45 → 1
`$var = r2 N`	Radice quadrata di N	$R = r2144 → 12
`$var = r3 N`	Radice cubica di N	$R = r327 → 3
`$var = log N,B`	Logaritmo di N in base B	$R = log100,10 → 2
`$var = map V,iMin,iMax,oMin,oMax`	Rimappatura lineare	$R = map512,0,1023,0,100 → 50
`$var = rnd MIN,MAX`	Numero casuale (TrueRandom hardware)	$R = rnd1,100 → 1..99

- *Le funzioni trigonometriche (sin, cos, tan) ricevono gradi, non radianti.*
- *rnd usa un sistema TrueRandom , non è un sistema pseudocasuale.*
- *map rimappa linearmente: map(valore, inMin, inMax, outMin, outMax).*

Specificatori di tipo risultato

Aggiungendo una lettera come ultimo token separato da spazio (o attaccato alla fine del numero) si forza il tipo del risultato:

Suffisso	Tipo	Esempio	Output
`i (default)`	int — parte intera	$A = 10 / 3 i	3
`f`	float — decimali	$A = 10 / 3 f	3.333333
`c`	char — carattere ASCII	$A = 65c	'A'
`s`	string — stringa	$A = 3.14 s	"3.14"

Esempi completi

```
$A = 10 + 5                  // A = 15 (int)
$B = $A * 2                  // B = 30
$C = r2144                   // C = 12 (radice di 144)
$D = (10 + 20) / 3 f         // D = 10.0 (float)
$E = 2@8                     // E = 256 (2 elevato a 8)
$F = sin90                   // F = 1
$G = rnd1,100                // G = numero casuale 1-99
$H = map512,0,1023,0,255     // H = rimappatura ADC → PWM
$I = log1000,10              // I = 3
$J = !0                      // J = 1 (NOT logico)
$K = ~255                    // K = -256 (NOT bitwise)
$A++                         // incrementa A
$A--                         // decrementa A
```

- *Un singolo carattere alfabetico usato come operando nelle espressioni $ viene automaticamente convertito nel suo codice ASCII. Es: $X = A → X = 65.*

4. Lettura Input — in

Il comando in legge un valore dall'hardware o dall'utente e lo assegna a una variabile. Si usa sempre all'interno di un'espressione $:

```
$VARNAME = in TIPO SORGENTE PIN_o_niente;
```

Tipo	Sorgente	Descrizione
`$var = in D PIN`	Pin / hardware	Legge pin digitale (0 o 1). Imposta il pin come INPUT.
`$var = in A PIN`	Pin / hardware	Legge pin analogico (0–1023, 10 bit).
`$var = in K`	default	Legge UN singolo carattere dalla Seriale O I2C (whichever first)
`$var = in KS`	Serial	Legge un carattere solo dalla porta Seriale
`$var = in KI`	I2C	Legge un carattere solo dall'I2C
`$var = in T`	default	Legge una riga di testo (fino a \n) dalla Seriale O I2C
`$var = in TS`	Serial	Legge una riga di testo solo dalla porta Seriale
`$var = in TI`	I2C	Legge una riga di testo solo dall'I2C

- *in K e in T sono bloccanti: l'esecuzione si ferma finché non arriva input. In K ha timeout 5 s, inT ha timeout 10 s.*
- *Prima di in K e in T viene automaticamente svuotato il buffer seriale per evitare di leggere caratteri residui.*

Esempi

```
$Stato = in D 13              // legge pin digitale 13
$Pot = in A 0                 // legge pin analogico A0 (0-1023)
$Tasto = in K                 // aspetta un tasto da seriale o I2C
$TastoSerial = in KS          // aspetta solo da seriale
$Riga = in T                  // aspetta riga di testo (fino a Invio)
$RigaI2C = in TI              // aspetta riga solo dall'I2C
```

```
// Esempio: menu interattivo
print "Premi 1 per LED on, 2 per LED off:\n"
$Scelta = in KS
if ($Scelta == 1) { out d 13 on; }
if ($Scelta == 2) { out d 13 off; }
```

```
// Esempio: lettura analogica con rimappatura
$Valore = in 14
$PWMval = map $Valore,0,1023,0,255
out a 9 $PWMval               // applica il valore al PWM hardware
```

5. Output Stampa — print / wait

print — Stampa testo e valori su seriale o I2C

Stampa sulla seriale o su un dispositivo I2C. Distingue due modalità in base alla presenza delle virgolette:

Contesto	Comportamento
`Dentro "X"`	Testo letterale. $NOME viene espanso col valore della variabile. Sequenze \n \t \r attive.
`Fuori X`	Valutato come espressione matematica tramite $NEO. Variabili, operatori, funzioni.
`Misto "X" Y "Z"`	Parti dentro e fuori virgolette si concatenano in un unico output.
`I2C @@ "X"`	print @INDIRIZZO@ contenuto — due @ obbligatori. Limite 32 byte.

Sequenze di escape valide dentro le virgolette

Sequenza	Carattere prodotto
`\n`	Nuova riga (LF, 0x0A)
`\t`	Tabulazione (TAB, 0x09)
`\r`	Carriage return (CR, 0x0D)
`\X`	Qualsiasi altro carattere X viene stampato letteralmente (es: \" stampa una virgoletta)

⚠ *La sintassi I2C e': print @INDIRIZZO@ "testo" con DUE simboli @: prima e dopo l'indirizzo.*

Esempi

```
print "Benvenuto!\n"
var T = 23
print "Temperatura: $T gradi\n"
$S = 10 + 20
print "Somma = " $S "\n"          // misto: testo + espressione
print $T * 2                       // fuori virgolette: stampa 46
print @0x08@ "out d 13 on"         // invia via I2C
```

wait — Pausa dell'esecuzione (millisecondi)

Sospende l'esecuzione per il numero di millisecondi indicato. Il parametro può essere un valore diretto o una variabile con $.

```
wait 1000                      // pausa 1 secondo
wait 500                       // pausa 500 ms
var T = 2000
wait $T                        // pausa 2 secondi da variabile
```

6. Blocchi e Funzioni — blk / vob

NEO-Shell permette di definire blocchi di codice riutilizzabili simili a funzioni. Un blocco viene definito con blk e salvato in una variabile. Può accettare parametri tramite i segnaposto vob dichiarati al suo interno.

`blk` — Definisce un blocco di codice (funzione)

Crea una variabile il cui valore è un intero blocco di codice racchiuso tra [ ]. Il blocco si richiama scrivendo semplicemente il suo nome come comando. Se contiene vob, accetta parametri alla chiamata.

Sintassi	Descrizione
`blk NOME = [ ... ]`	Crea blocco con nome diretto
`blk $NOMEVAR = [ ... ]`	Crea blocco con nome preso dal valore di NOMEVAR (indirezione)

```
blk Saluta = [ print "Ciao dal blocco!\n" ]

Saluta;                          // richiama il blocco
```

`vob` — Parametro formale di un blocco (Variable Of Block)

Si dichiara all'interno di un blk per indicare che accetta un parametro. Il separatore opzionale (tra virgolette doppie dopo il nome) specifica con quale carattere/stringa si separa il parametro successivo. Default: spazio.

Sintassi	Descrizione
`vob NOME`	Parametro separato da spazio (default)
`vob NOME ","`	Parametro separato da virgola
`vob NOME "7"`	Parametro separato da un numero
`vob NOME "%"`	Parametro separato da percentuale

⚠ *se utilizzare il ";" ce il rischio che l'interprete possa prendere il valore successivo come comando da eseguire e non come valore da associare al vob successivo.*

L'ultimo vob di un blocco raccoglie sempre tutti i caratteri rimanenti, indipendentemente dal separatore.

Esempio: blocco con due parametri (separatore spazio)

```
blk Somma = [
    vob A;
    vob B;
    $R = $A + $B;
    print "Risultato: " $R "\n"
]
Somma 10 20                    // A=10, B=20
Risultato: 30
```

Esempio: separatore personalizzato

```
blk Info = [
    vob Nome ","                            // primo param fino alla virgola
    vob Eta                                 // secondo param: resto della stringa
    print "Nome: " $Nome " Eta: " $Eta "\n"
]
```

```
Info Mario,25                               // Nome=Mario, Eta=25
```

7. Controllo del Flusso — if / elseif / else

NEO-Shell supporta strutture condizionali complete. Le condizioni usano operatori di confronto e operatori logici. I confronti funzionano sia su numeri (confronto numerico con tolleranza floating point) che su stringhe (confronto lessicale).

Operatore	Significato	Note
`==`	Uguale	Numerico (tolleranza 0.0001) o lessicale
`!=`	Diverso	Numerico o lessicale
`>`	Maggiore di	
`<`	Minore di	
`>=`	Maggiore o uguale	
`<=`	Minore o uguale	
`&&`	AND logico	Combina più condizioni
`||`	OR logico	Combina più condizioni

`if` — Esecuzione condizionale

```
if (condizione) { istruzioni }
```

```
var X = 10
if ($X > 5) { print "X > 5\n" }
if ($X >= 10 && $X < 20) { print "tra 10 e 19\n" }
if ($X == 10 || $X == 20) { print "10 o 20\n" }
```

`elseif` — Ramo condizionale alternativo

Deve seguire immediatamente un if (o un altro elseif). Viene eseguito solo se tutti i rami precedenti erano falsi.

```
elseif (condizione) { istruzioni }
```

`else` — Ramo di default (altrimenti)

```
else { istruzioni }
```

Esempio completo — catena if/elseif/else

```
var Voto = 75

if ( $Voto >= 90 ) { print "Ottimo!\n" }

elseif ( $Voto >= 70 ) { print "Buono\n" }
elseif ( $Voto >= 60 ) { print "Sufficiente\n" }

else { print "Insufficiente\n" }
```

8. Cicli — for / while / loop / switch

for — Ciclo con inizializzazione, condizione e incremento

Ciclo classico con tre parti separate da ; nella condizione. La variabile viene dichiarata automaticamente con var. Supporta break, continue, return.

```
For ( i = 0; $i < 10; $i++) { istruzioni }
```

⚠ *Il for usa espressioni di calcoli semplici: gli operatori logici && e || NON funzionano nella condizione del for. Per condizioni complesse usare un if interno con break.*

```
// Conta da 0 a 9
for ( i = 0; $i < 10; $i++) { print $i "\n" }
```

```
// Ciclo con decremento
for ( k = 10; $k > 0; $k--) {
  out d 13 on
  wait 100
  out d 13 off
  wait 100
}
```

while — Ciclo con condizione di ingresso

Ripete il blocco finché la condizione è vera. La condizione viene valutata prima di ogni iterazione.

```
while (condizione) { istruzioni }
```

```
var N = 0
while ( $N < 5 ) { print $N "\n" ; $N++ }
```

loop — Ciclo infinito

Esegue il blocco all'infinito senza condizione. Per uscire usare break o return. Ideale per programmi di monitoraggio continuo.

```
loop { istruzioni }
```

```
var C = 0
loop { $C++ ; print "Tick " $C "\n" ; wait 100 ; if ($C >= 60) { break } }

//Esce dopo 60 interazioni
```

switch — Selezione multipla su un valore

Confronta il valore di una variabile con una serie di case. Se nessun case corrisponde, esegue il blocco DEFAULT (se presente). Ogni case normalmente termina con break per evitare di eseguire i case successivi (fall-through).

```
switch ($VARIABILE) {
  case VALORE1 { istruzioni; break }
  case VALORE2 { istruzioni; break }
  DEFAULT      { istruzioni }
}
```

Esempio completo

```
var Giorno = 3
switch ($Giorno) {
  case 1 { print "Lunedì\n";    break }
  case 2 { print "Martedì\n";   break }
  case 3 { print "Mercoledì\n"; break }
  case 4 { print "Giovedì\n";   break }
  DEFAULT { print "Weekend\n" }
}
```

9. Segnali di Controllo — break / continue / return

break — Esce dal ciclo o dallo switch corrente

Interrompe immediatamente l'esecuzione del ciclo (for, while, loop) o dello switch in cui si trova. L'esecuzione riprende dall'istruzione successiva al blocco.

```
var I = 0
while (1 == 1) { $I++ ; if ($I >= 5){ break } }

print "Uscito con I = " $I "\n"   // stampa: 5
```

continue — Salta all'iterazione successiva del ciclo

Interrompe l'iterazione corrente e passa alla successiva. In un for, esegue l'incremento prima di tornare alla condizione.

```
for ( k = 0; $k < 10; $k++) { if ($k == 5) { continue };  print $k " " }
// salta quando k=5

// Output: 0 1 2 3 4 6 7 8 9
```

return — Esce dal blocco blk corrente (con valore opzionale)

Termina l'esecuzione del blocco blk corrente e restituisce il controllo al chiamante. Può opzionalmente restituire un valore che viene salvato nella variabile di sistema $return.

Sintassi	Descrizione
`return;`	Esce dal blk senza valore di ritorno
`return valore;`	Esce dal blk e salva 'valore' in $return
`return $EXPR;`	Esce dal blk e salva il valore di $EXPR in $return

⚠ *return e' scoped al blk: il flag shouldReturn viene resettato automaticamente dopo il termine dell'esecuzione del blocco. Un return dentro un blk NON si propaga all'esterno — il codice chiamante continua normalmente.*

```
blk Massimo = [ vob A ; vob B ; if ($A > $B) { return $A }; return $B ]

Massimo 10 20
print "Max = " $return "\n"     // stampa: Max = 20
```

10. Porte I/O — out

Il comando out gestisce tutte le uscite hardware. Il tipo di uscita è la lettera immediatamente dopo out (d, a, t, s, p). I valori parametro possono essere costanti o variabili $.

Comando	Descrizione	Esempio
`out d PIN on`	Output digitale ALTO	out d 13 on
`out d PIN off`	Output digitale BASSO	out d 13 off
`out d PIN hi`	Output digitale ALTO (alias)	out d 13 hi
`out d PIN lo`	Output digitale BASSO (alias)	out d 13 lo
`out d PIN 1`	Output digitale ALTO (numerico)	out d 13 1
`out a PIN val`	PWM hardware analogWrite (0-255)	out a 9 128
`out t PIN freq`	Tone: suono continuo su PIN	out t 8 440
`out t PIN freq dur`	Tone con durata in millisecondi	out t 8 440 500
`out t PIN 0`	Disattiva il tone (noTone)	out t 8 0
`out s PIN gradi`	Servo: posiziona a N gradi (0-180)	out s 6 90
`out p PIN freq duty`	PWM software: freq Hz, duty %	out p 5 1000 50
`out p PIN freq duty dur`	PWM software con durata ms	out p 5 1000 50 3000
`out p PIN 0 0`	Stop PWM software su PIN	out p 5 0 0

out p genera PWM software: supporta fino a 5 segnali simultanei su qualsiasi pin. La frequenza e il duty cycle sono completamente liberi. Con freq=0 o duty=0 il segnale viene fermato.

Uso delle variabili nei parametri

```
var PIN = 13 , Stato = 110
out d $PIN on                  // usa PIN da variabile
out a 9 $Stato                 // usa Stato come valore PWM
```

Esempi pratici

```
// Blink LED
loop { out d 13 on ; wait 500 ; out d 13 off ; wait 500 }
```

```
// Servo sweep
for ( pos = 0 ; $pos <= 180 ; $pos++ ) { out s 13 $pos; wait 15 }
```

```
// PWM software 2kHz, 30% duty, 5 secondi
out p 5 2000 30 5000;
```

11. EEPROM — Memoria Programma Persistente

La EEPROM permette di salvare un programma NEO-Shell che sopravvive allo spegnimento. Le righe sono numerate da 1. La riga 1 è riservata al marcatore boot. Il programma viene eseguito con eeprun.

eepset — Scrittura, modifica e manipolazione righe EEPROM

Sintassi	Descrizione
`eepset`	Modalita' editor interattivo — digita righe, termina con 'end'
`eepset N testo`	Scrive/sostituisce 'testo' nella riga N
`eepset +N testo`	Inserisce 'testo' come nuova riga in posizione N (sposta le successive)
`eepset +N`	Inserisce una riga vuota nella posizione N
`eepset => A B`	Scambia il contenuto della riga A con la riga B
`eepset => A@B C@D`	Scambia il gruppo di righe A-B con il gruppo C-D
`eepset <= A B`	Come =>, ma nella direzione inversa (B -> A)
`$ VAR = eepset N`	Legge la riga N dalla EEPROM e la salva in VAR

> *eepset sostituisce automaticamente le variabili $NOME nel testo da scrivere. Per scrivere un $ letterale senza espansione usare $$NOME — il doppio $$ viene ridotto a $ singolo.*
> *La lettura $ VAR = eepset N usa la sintassi del comando $ (non var). Esempio: $ Riga = eepset 3;*

```
eepset;                          // avvia editor interattivo
```

L'editor interattivo mostra feedback in tempo reale per ogni riga inserita:

```
  TESTO << Used: 120 Free: 903 byte >>
```

Per uscire dall'editor digitare 'end'. L'editor accetta input sia dalla porta Seriale che dall'I2C.

In tal caso e possibile modificare le singole stringhe del codice inserendo prima il numero della liga interessata a modificare e poi si inserisce il nuovo testo o comando.

```
eepset 1 boot                  // riga 1 = marcatore boot
eepset 2 var LED = 13          // scrive codice in riga 2
eepset +3 print "inserita"   // inserisce nuova riga in pos. 3
eepset +5                      // inserisce riga vuota in pos. 5
eepset => 2 5                  // scambia righe 2 e 5
eepset <= 2 5                  // scambia in direzione opposta
eepset 2 print $$Var           // scrive letteralmente: print $Var
$ Riga = eepset 2              // legge riga 2 in variabile Riga
```

eeplist — Visualizzazione del contenuto EEPROM

Sintassi	Descrizione
`eeplist`	Mostra tutte le righe con numero e contenuto
`eeplist N`	Mostra solo la riga N
`eeplist N@M`	Mostra le righe da N a M incluse
`eeplist N,M,K`	Mostra le righe N, M e K (lista arbitraria)

```
eeplist                        // visualizza tutto
eeplist 1@5                    // visualizza righe 1-5
eeplist 2,4,6                  // visualizza righe 2, 4, 6
```

eeprun — Esecuzione del programma EEPROM

Sintassi	Descrizione
`eeprun`	Esegue TUTTE le righe (la riga 1 'boot' viene saltata)
`eeprun N`	Esegue solo la riga N
`eeprun N>`	Esegue dalla riga N fino alla fine
`eeprun N@M`	Esegue le righe dalla N alla M incluse
`eeprun N,M,K`	Esegue le righe N, M e K in quell'ordine

```
eeprun                          // esegui programma
eeprun 5                        // esegui solo riga 5
eeprun 2@8                      // esegui righe 2-8
eeprun 3>                       // esegui dalla riga 3 in poi
eeprun 2,5,7                    // esegui righe 2, 5 e 7
```

eepcls — Cancellazione righe EEPROM

Cancella righe dalla EEPROM. Esistono DUE modalità di cancellazione:

Modalità	Sintassi	Comportamento
`Svuota (default)`	eepcls N	Sostituisce la riga con 'Empty' — la riga rimane ma è vuota
`Elimina fisicamente`	eepcls N!	Elimina completamente la riga — le righe successive si compattano
`Range svuota`	eepcls N@M	Svuota le righe da N a M
`Range elimina`	eepcls N!@M!	Elimina fisicamente le righe da N a M
`Lista mista`	eepcls N,M!,K	N svuotata, M eliminata, K svuotata
`Formato completo`	eepcls	Azzera TUTTA la EEPROM (byte per byte)

⚠ *eepcls senza ! svuota la riga (scrive 'Empty'). Con ! la elimina fisicamente, ricompattando le righe successive. I numeri di riga cambiano dopo un'eliminazione.*

```
eepcls                          // format totale EEPROM
eepcls 5                        // riga 5 → 'Empty'
eepcls 5!                       // riga 5 eliminata fisicamente
eepcls 2@4                      // righe 2,3,4 → 'Empty'
eepcls 2!@4!                    // righe 2,3,4 eliminate
```

eepmem — Statistiche memoria EEPROM

Mostra i byte usati e i byte liberi rimasti nella EEPROM.

```
eepmem                          // es: MEM Bytes used 120 / MEM Bytes free 903
```

12. Protocollo I2C — iset / ifind

NEO-Shell opera simultaneamente come MASTER e SLAVE I2C. Come slave risponde all'indirizzo default 0x12. Come master può inviare comandi ad altri dispositivi. Il limite per singolo messaggio è 32 byte.

iset — Imposta l'indirizzo I2C target per le trasmissioni

Cambia l'indirizzo I2C di destinazione usato da print @...@. Accetta indirizzo in formato esadecimale (0xNN) o decimale. Reinizializza il bus Wire con il nuovo indirizzo slave.

Sintassi	Descrizione
`iset 0x08`	Imposta target su 0x08 (default NEO)
`iset 0x12`	Imposta target su 0x12
`iset $VAR`	Usa il valore della variabile come indirizzo

```
iset 0x08                       // cambia target a 0x08
```

Indirizzi validi: 0x08 – 0x77. Fuori da questo range viene emesso errore 'Addr Invalid'.

ifind — Scansione del bus I2C

Senza parametri esegue una scansione completa (0x08–0x77) e stampa tutti i dispositivi trovati. Con un indirizzo come parametro verifica solo quel dispositivo e imposta la variabile di sistema $I2C (1=trovato, 0=non trovato).

Sintassi	Descrizione
`ifind`	Scansione completa — stampa tutti gli indirizzi trovati
`ifind 0x08`	Verifica se l'indirizzo 0x08 esiste
`ifind $VAR`	Verifica l'indirizzo contenuto in $VAR

Quando viene cettacciato un indirizzo I2C , il neo shell rilascia una variabile "$I2C" se l'indirizzo esiste e viene agganciato allora il contenuto sarà 1 (TRUE) in caso contrario presentarà 0 (FALSE)

```
ifind                           // scansione completa
ifind 0x08                      // verifica 0x08
if ($I2C == 1) { print "Dispositivo trovato!\n"} //se esiste, stamperà il testo citato
```

13. Variabili di Sistema

NEO-Shell gestisce alcune variabili speciali che vengono aggiornate automaticamente dal sistema durante l'esecuzione.

Variabile	Tipo	Descrizione
`$return`	qualsiasi	Contiene il valore restituito dall'ultimo blk con 'return VALORE'
`$I2C`	int (0/1)	Impostata da ifind: 1 se il dispositivo e' trovato, 0 altrimenti
`$NEOin`	string	Contiene il testo del comando PRECEDENTE (aggiornato prima di eseguire il nuovo)
`$NEO`	calcolata	Variabile temporanea interna usata da print per valutare espressioni fuori da virgolette

$NEOin viene aggiornato prima dell'esecuzione del comando corrente: contiene sempre il comando immediatamente precedente, non quello appena lanciato.

Esempi

```
// Uso di $return
blk Rad = [ vob X; return r2$X ]
Rad 144
print "Radice = " $return "\n"    // stampa: 12
```

```
// Uso di $I2C
ifind 0x08
if ($I2C == 1) {
  print @0x08@ "sei collegato al terminale NEO-Shell!\n"
}
```

```
// Uso di $NEOin
print "Ultimo comando: " $NEOin "\n"
```

14. Comandi Speciali — boot / Empty

boot — Marcatore di avvio automatico EEPROM (riga 1)

Non è un comando eseguibile, ma un marcatore speciale che deve stare nella riga 1 della EEPROM. All'accensione, NEO-Shell controlla la riga 1: se contiene boot, esegue automaticamente le righe successive.

Sintassi riga 1	Comportamento all'avvio
`boot`	Esegue tutte le righe dalla 2 in poi (equivale a eeprun)
`boot N`	Esegue dalla riga N in poi
`boot N@M`	Esegue le righe da N a M
`boot N,M,K`	Esegue le righe N, M e K

Il confronto 'boot' nella riga 1 è case-insensitive: BOOT, Boot, boot funzionano tutti.

```
// Programma LED lampeggiante all'avvio:
eepset 1 boot
eepset 2 var LED = 13
eepset 3 loop { out d $LED on ; wait 500 ; out d $LED off ; wait 500 }
```

Empty — Marcatore di riga vuota nella EEPROM

Non è un comando eseguibile. Viene scritto automaticamente dall'interprete quando una riga viene cancellata con eepcls senza il modificatore ! (cancellazione logica). L'interprete ignora silenziosamente tutte le righe marcate Empty durante eeprun.

```
eepcls 2                        // riga 2 diventa 'Empty'
eeplist 2                       // mostra: 2 > Empty
eeprun                          // riga 2 viene saltata automaticamente
```

Appendice A — Messaggi di Sistema (MSGbox)

NEO-Shell usa una libreria interna di messaggi numerati per comunicare stati, errori e conferme. Questi appaiono automaticamente sul monitor seriale durante l'uso. La tabella completa:

N°	Messaggio seriale	Quando appare
0	Logic Components(C) / NEO-Shell Version I / Copyright...	All'avvio del dispositivo (banner)
1	>>	Prompt interattivo (pronto per il prossimo comando)
2	Run	Inizio esecuzione eeprun
3	Stop	Fine / stop esecuzione
4	Error	Errore generico di formato o sintassi
5	Ok	Operazione riuscita (generico)
6	Wait	Sistema in attesa
7	!	Avviso/warning generico
8	?	Richiesta input o indecisione
9	Cleared	Memoria variabili o EEPROM azzerata
10	Saved	Dati salvati correttamente (es: dopo eepset)
11	Format Type?	Formato non riconosciuto in un blk
12	Typo	Errore di battitura / comando non riconosciuto
13	Mem Empty!	Memoria variabili vuota
14	Mem Tall!	Memoria variabili piena (256 variabili raggiunto)
15	###	Barra vanzamento formattazione EEPROM
16	EEProm Empty!	EEPROM vuota (nessuna riga)
17	EEProm Tall!	EEPROM piena
18	MEM Bytes used	Prefisso per il conteggio byte usati (eepmem)
19	MEM Bytes free	Prefisso per il conteggio byte liberi (eepmem)
20	Not Exist!	Variabile o riga EEPROM non trovata
21	End!	Fine esecuzione boot o eeprun
22	ok	Trasmissione I2C riuscita (i2cSendData)
23	Error: NACK Addr	I2C: il destinatario non ha riconosciuto l'indirizzo
24	Error: NACK Data	I2C: il destinatario non ha accettato i dati
25	Error: Timeout	I2C: timeout nella trasmissione
26	Error: Data Long	I2C: dato troppo lungo (>32 byte)
27	Addr: 0x	Prefisso visualizzazione indirizzo I2C
28	No devices	ifind: nessun dispositivo I2C trovato
29	Scanning...	ifind: scansione bus in corso
30	0x	Prefisso indirizzo trovato durante ifind
31	Addr Invalid	Indirizzo I2C fuori range (valido: 0x08-0x77)

15. Riferimento Rapido Completo

Comando / Sintassi	Utilizzo	Categoria
`var X = valore`	Dichiara / aggiorna variabile	Variabili
`var A=1, B=2, C=3`	Dichiarazione multipla	Variabili
`killvar [X / A,B / A@Z]`	Elimina variabile/e	Variabili
`allvar [X / A,B / A@Z]`	Visualizza variabile/e	Variabili
`$ X = expr`	Calcolo matematico, risultato in X	Espressioni
`$ X++ / $ X--`	Incremento / decremento	Espressioni
`$ X = A > B (clamp)`	Clamp: ritorna B se A>B, altrimenti A	Espressioni
`$ X = A < B (clamp)`	Clamp: ritorna B se A<B, altrimenti A	Espressioni
`$ X = rndMIN,MAX`	Numero casuale hardware TrueRandom	Espressioni
`$ X = r2N / r3N`	Radice quadrata / cubica	Espressioni
`$ X = sinN / cosN / tanN`	Trig in gradi	Espressioni
`$ X = logN,B`	Logaritmo in base B	Espressioni
`$ X = mapV,i0,i1,o0,o1`	Rimappatura lineare	Espressioni
`$ X = A@B`	Potenza: A elevato a B	Espressioni
`$ X = !V / ~V`	NOT logico / NOT bitwise	Espressioni
`$ X = inDPIN`	Legge pin digitale (0/1)	Input
`$ X = inAPIN`	Legge pin analogico 0-1023	Input
`$ X = inK[S/I]`	Legge un carattere (S=Serial, I=I2C)	Input
`$ X = inT[S/I]`	Legge una riga testo (S=Serial, I=I2C)	Input
`print "testo"`	Stampa su seriale	Output
`print @0xNN@ "testo"`	Invia via I2C (due @ obbligatori)	Output
`wait ms`	Pausa in millisecondi	Output
`blk N = [ ... ]`	Definisce blocco/funzione N	Blocchi
`vob NOME ["sep"]`	Parametro formale in un blk	Blocchi
`N params`	Chiama blocco N con parametri	Blocchi
`if (cond) {...}`	Esecuzione condizionale	Flusso
`elseif (cond) {...}`	Condizione alternativa	Flusso
`else {...}`	Ramo else	Flusso
`for (init;c;inc) {}`	Ciclo for classico	Cicli
`while (cond) {}`	Ciclo while	Cicli
`loop {}`	Ciclo infinito	Cicli
`switch ($X) { case ... }`	Selezione multipla	Cicli
`break`	Esce dal ciclo / switch	Controllo
`continue`	Prossima iterazione del ciclo	Controllo
`return [valore]`	Esce dal blk, valore opzionale in $return	Controllo
`out d PIN on/off/hi/lo`	Uscita digitale	I/O
`out a PIN val`	PWM hardware analogWrite 0-255	I/O
`out t PIN freq [dur]`	Tone (suono su pin)	I/O
`out s PIN gradi`	Servo motore 0-180	I/O
`out p PIN f d [dur]`	PWM software multi-canale (max 5)	I/O
`eepset N testo`	Scrive riga EEPROM	EEPROM

`eepset +N [testo]`	Inserisce riga (testo opzionale)	EEPROM
`eepset => A B`	Scambia righe A e B	EEPROM
`eepset <= A B`	Scambia in direzione opposta	EEPROM
`$ X = eepset N`	Legge riga N in variabile X	EEPROM
`$$NOME in eepset`	Scrive $NOME letterale (escape del dollaro)	EEPROM
`eeplist [N / N@M / N,M]`	Visualizza righe EEPROM	EEPROM
`eeprun [N / N> / N@M]`	Esegue righe EEPROM	EEPROM
`eepcls [N / N! / N@M]`	Svuota (default) o elimina fisicamente (con !)	EEPROM
`eepmem`	Statistiche byte EEPROM	EEPROM
`iset 0xNN`	Imposta indirizzo I2C target (0x08-0x77)	I2C
`ifind [0xNN]`	Scansione bus / verifica -> $I2C	I2C
`$return`	Valore dall'ultimo return in un blk	Sistema
`$I2C`	1=trovato / 0=non trovato (aggiornato da ifind)	Sistema
`$NEOin`	Testo del comando PRECEDENTE	Sistema
`$NEO`	Var temporanea interna (usata da print)	Sistema
`boot [N / N@M / N,M,K]`	Avvio automatico EEPROM (riga 1)	Speciale
`Empty`	Marcatore riga cancellata logicamente	Speciale

Logic Components Corp · NEO-Shell® Version I · By Fogli Luca · 2024 – 2025